

LaTeX 编译与中文字体疑难解析

——计算机专业学习笔记

你的名字

2026 年 6 月 4 日

目录

1 从源码到 PDF: LaTeX 的“翻译”流程

我们可以把一份 .tex 源码想象成一封用世界通用语写的信，而 L^AT_EX 引擎则是一位翻译官。信件内容完全一样，但交给不同的翻译官，他们会用不同的方式表达出来，最终生成一份精美的 PDF 文件。

基本流程如下：

1. 用任意文本编辑器（vim、nano、VS Code 等）写一个 .tex 文件。
2. 在终端调用某个 L^AT_EX 引擎进行编译。
3. 引擎读入 .tex 源码，处理其中的命令和环境，输出 .pdf。

最简单的英文文档

```
\documentclass{article}
\begin{document}
Hello, \LaTeX!
\end{document}
```

编译命令：pdf_latex hello.tex

支持中文的文档

```
\documentclass{ctexart} % 或者 \documentclass[UTF8]{ctexart}
\begin{document}
你好, \LaTeX! 计算机学习快乐!
\end{document}
```

编译命令：xe_latex hello.tex

2 认识三位翻译官：pdfTeX、XeTeX、LuaTeX

这三位“翻译官”各有所长，理解它们的区别是举一反三的关键。

pdfTeX（老牌翻译官）

最传统的引擎，直接输出 PDF。它最擅长处理英文和传统 TeX 字体，对 Unicode 的支持非常有限，处理中文时就像一个只会说方言的老翻译官——需要借助 CJK 宏包和各种专门的字体包，稍有不慎就会“翻译失败”。

XeTeX（现代多语种翻译官）

一位精通多国语言的现代翻译官。它原生支持 UTF-8 编码，能直接调用你操作系统里安装的任何字体（如宋体、黑体、Noto 等）。处理中文时，XeTeX 是**首选**。

LuaTeX (全能程式翻译官)

XeTeX 的“升级版”，同样支持 Unicode 和系统字体，而且内部嵌入了一个 Lua 脚本解释器，允许用编程方式深度定制排版逻辑。功能非常强大，但日常使用中，XeTeX 已经足够。

关键结论

简单来说：

- 纯英文文档、老式论文模板：用 `pdflatex`
- 带中文、多语种的文档：用 `xelatex` (推荐) 或 `lualatex`

3 中文排版为何首选 XeLaTeX ?

这就要讲到字体处理的**编码模型**了。

- **pdfTeX 的世界**：传统 TeX 字体每个字符占 1 个字节（最多 256 个字符位置）。英文绰绰有余，但中文有上万个汉字，必须把字体分割成多个小字库，还要通过 `.tfm` 文件和映射关系来定位。一旦某个字体包没装好，就会出“fandol 不可用”这类错误。
- **XeTeX 的世界**：直接使用 **UTF-8 编码**，每个字符有全局唯一的编码，字体的选择直接交给操作系统。只要你的系统里安装了中文字体，XeTeX 就能正确显示。这就好比打电话：pdfTeX 需要提前查内部黄页，而 XeTeX 直接拨号。

因此在实践中，`ctex` 宏包搭配 XeTeX 可以省去无数字体配置的烦恼。

4 实战排错：fandol 字体缺失错误

你是不是在终端敲了 `pdflatex fedora-install-crewai-note.tex`，然后看到了下面这样的错误？

```
! Critical Class ctexart Error: CTeX fontset `fandol' is unavailable in
(ctexart)                                current mode.
```

错误翻译：“ctexart 文档类严重错误：在当前模式下，CTeX 字体集 fandol 不可用。”

4.1 为什么会这样？

因为你在用 `pdflatex` (老翻译官) 编译一个使用了 `ctexart` 的中文文档。`ctexart` 在 pdfTeX 模式下默认选择 **fandol** 字体集，它需要安装对应的 `texlive-fandol` 包，但你的 Fedora 系统上并没有装这个包。

更本质的原因是：fandol 是为 pdfTeX 准备的一套 Type1 中文字体，而很多 TeX 发行版的最小安装中并不包含它。

4.2 两种解决方案

方案一：换个翻译官（推荐）

直接改用现代引擎 xelatex：

```
xelatex fedora-install-crewai-note.tex
```

优点：

- 彻底避免 pdfTeX 字体问题
- 直接利用系统字体，兼容各种语言文字
- 编译速度通常也更快（中文部分）

方案二：给老翻译官配字典（备选）

安装缺失的 fandol 字体包：

```
sudo dnf install texlive-fandol
```

然后再用 pdfflatex 编译。如果想用其他字体集，也可以修改文档类选项：

```
\documentclass[fontset=ubuntu]{ctexart} % 使用文泉驿字体，需提前安  
装
```

4.3 我是怎么诊断出来的？

1. **看日志**：.log 文件里明明白白写着 fandol 不可用。
2. **查引擎**：确认一下自己是不是误用了 pdfflatex。
3. **试切换**：有条件的优先换 xelatex，一般立刻解决问题。
4. **搜包名**：实在要用 pdfflatex 时，用 `dnf search` 搜索 fandol 或 texlive-lang-chinese 来补装字体。

5 ctex 宏包的字体集机制

ctex 文档类（ctexart, ctexbook, ctexrep）会根据**编译引擎**自动选择一套中文字体集。这套机制就像一个聪明的管家：

引擎	默认字体集	背后的字体调用
pdfLaTeX	fandol (未安装则报错)	CJK 宏包 + fandol Type1 字体
XeLaTeX	系统字体 (如宋体、黑体)	fontspec 宏包 + 操作系统字体
LuaLaTeX	系统字体	fontspec 宏包 + 操作系统字体

你可以用 `fontset` 选项手动指定字体集，例如：

```
\documentclass[fontset=windows]{ctexart} % 需要 Windows 系统字体
\documentclass[fontset=mac]{ctexart} % 需要 macOS 系统字体
\documentclass[fontset=fandol]{ctexart} % 强制使用 fandol
```

在 Fedora 等 Linux 系统下，最省心的做法就是**不指定** `fontset`，直接用 `xelatex`，让管家自动调用系统中已安装的文泉驿、Noto 等开源字体。

6 另一个常见陷阱：\end 不能在 \texttt 里裸用

6.1 事故现场

在编写这份笔记时，我们遇到了一个奇怪的编译错误，按回车后还无限循环：

```
! Missing \endcsname inserted.
<to be read again>
      \aftergroup
1.270 ...nning use of ...} → 缺少右花括号或 \texttt{\end}
```

出错的这一行想表达的意思其实是：“缺少右花括号或 `\end`”，但在代码里我们直接写了：

```
... 缺少右花括号或 \texttt{\end}
```

6.2 为什么会爆炸？

在 LaTeX 中，`\end` 是一个控制序列，它的作用是结束当前环境（如 `\end{document}`、`\end{itemize}`）。当你把它放进 `\texttt{...}` 这类命令的参数里时，LaTeX 会先尝试展开（执行）这个控制序列，而不是把它当成普通文字打印出来。因为此时并没有一个合理的环境可以结束，内部机制就会彻底混乱，抛出 `Missing \endcsname` 错误。

简单来说：**控制序列 `\end` 在参数里“越狱”了，破坏了 LaTeX 的正常语法结构。**

6.3 安全打印控制序列的方法

1. 用 `\verb`（最推荐）

`\verb` 会原样输出其中的所有字符，不解释任何命令。缺点是它不能出现在其他命令的参数中，所以通常需要单独使用。例如：

```
... 缺少右花括号或 \verb|\end|，找一个花括号匹配插件...
```

2. 用 `\texttt{\textbackslash end}`

先用 `\textbackslash` 输出一个反斜杠符号，再跟上普通文字 `end`，这样就不会触发控制序列了。

3. 用 `\string`

`\string` 会让紧随其后的控制序列丧失功能，变成普通字符序列。例如：

```
\texttt{\string\end}
```

6.4 举一反三

这个原则不仅适用于 `\end`，对所有 LaTeX 特殊字符（如 `\begin`、`{}`、`%`、`#` 等）都成立。当你能在普通文字（或等宽字体）中显示它们时，必须使用对应的转义命令或 `\verb`。养成习惯后，这类“诡异”的编译错误一眼就能看穿。

7 搭建高效的 LaTeX 编译 workflow

7.1 工具选择

- **纯命令行**：vim/emacs + 终端，适合极客
- **现代编辑器**：VS Code 安装 LaTeX Workshop 插件，自动编译和预览
- **专用 IDE**：TeXstudio、TeXmaker, Fedora 下 `sudo dnf install texstudio`

7.2 编译自动化

如果文档有目录、交叉引用或参考文献，需要多次编译。可以用自动化工具：

- **latexmk**：智能判断需要编译几次，例如
`latexmk -pdf -xelatex mydoc.tex`
它会自动调用 `xelatex` 并运行必要的 `bibtex` 等。
- **编写 Makefile 或脚本**，适合大项目。

8 举一反三：常见编译错误与调试心法

掌握了上面这些，你就能解决一大类“找不到字体/宏包”的问题。这里再总结一个通用排错框架。

遇到错误不要慌，三问自己

1. 引擎用对了吗？

中文文档用 `xelatex`，带特定字体要求的模板可能只能用 `pdflatex`。

2. 宏包/字体装了吗？

类似 `! LaTeX Error: File 'xxx.sty' not found.` 就是宏包缺失；`fandol` 不可用则是字体缺失。Fedora 下用 `dnf search` 搜索安装。

3. 源码本身有语法错误吗？

少个花括号、命令拼写错误、数学模式内外混乱都会引发连锁报错。顺着第一个错误往上找。

8.1 常见错误速查

- `Undefined control sequence` → 命令拼写错误或宏包未加载。
- `Misplaced alignment tab character &` → 表格/矩阵的 `&` 写在了外面，检查是否漏了环境。
- `File ended while scanning use of ...` → 缺少右花括号或 `\end`，找一个花括号匹配插件可快速定位。
- `Font ... not found` → XeLaTeX 下缺少系统字体，安装对应字体或换字体集。

8.2 一个更直观的类比

把 LaTeX 编译想象成盖房子：

- `.tex` 源文件是**建筑设计图**。
- 各种宏包和字体是**建筑材料**。
- 编译引擎是**施工队**（pdfTeX 施工队、XeTeX 施工队）。
- 报错就是施工过程中发现图纸矛盾，或材料缺货。

图纸没问题、材料齐全、找对施工队，房子（PDF）自然就盖好了。

9 结语

至此，你不仅知道了如何创建和编译 LaTeX 文档，还深入理解了编译引擎的差异、字体处理的底层逻辑，以及像 `\end` 这样的控制序列在参数中使用的禁忌。以后再遇到类似错误，就不需要死记硬背，而是能够分析——切换引擎、补装材料或检查特殊字符，从而游刃有余。

——编译愉快，学习顺利！